

DATA SCIENCE PROJECT EVALUATION

Student: Maureen Petterson

Project: Predicting the 2016 Presidential Election Using County-Level Demographics

Evaluation Date: November 03, 2025

Assignment: End-to-end Data Science Workflow

Student Level: Beginner (9-month Data Science Program)

EXECUTIVE SUMMARY

This is an exceptionally strong project for a beginner data science student. The work demonstrates a complete understanding of the full data science workflow, from data cleaning through modeling to interpretation. The student shows impressive code organization with modular helper functions, employs multiple modeling approaches (Linear Regression, Random Forest, Gradient Boosting) with proper validation techniques, and creates excellent visualizations including choropleth maps. The comprehensive README is publication-quality.

Minor deductions for: (1) a code error in one notebook, (2) limited in-notebook narrative documentation, and (3) missing dependency management files (requirements.txt) and installation instructions, which impacts reproducibility.

RECOMMENDED GRADE: A- (91/100)

DETAILED SCORING

Technical Data Science Skills (38/40 pts)

1. Problem Understanding & Approach (8/8 pts)

- ✓ Excellent problem framing: predicting county-level voting percentages
- ✓ Clear understanding that regression is appropriate for continuous outcomes
- ✓ Thoughtful justification for using demographics only
- ✓ Appropriate train/test split methodology

2. Data Exploration & Understanding (8/8 pts)

- ✓ Comprehensive EDA with correlation heatmaps
- ✓ Scatter matrices to identify multicollinearity
- ✓ Histogram comparisons between Trump/Clinton counties
- ✓ Choropleth maps for geographic patterns

3. Data Preprocessing & Feature Engineering (7/8 pts)

- ✓ Proper handling of missing data (3 counties dropped, justified)
- ✓ Created meaningful features (vote percentages, turnout rates)
- ✓ One-hot encoding for categorical variables
- ✓ Removed highly correlated features to address multicollinearity
- Normalization vs standardization mentioned but not explored in notebooks

4. Model Development & Validation (8/8 pts)

- ✓ Three models: Linear Regression, Random Forest, Gradient Boosting
- ✓ Proper 75/25 train/test split
- ✓ KFold cross-validation (5 folds) implemented correctly
- ✓ Feature reduction iterations tested
- ✓ Statistical significance testing with statsmodels

5. Results Interpretation (7/8 pts)

- ✓ Clear interpretation of model performance ($R^2 = 0.95-0.96$)
- ✓ Analysis of feature importance and coefficients
- ✓ County-level accuracy (96-97% correct predictions)
- ✓ Thoughtful discussion of limitations
- No residual analysis or assumption checking for linear regression

Code Quality (20/25 pts)

1. Code Organization & Structure (7/8 pts)

- ✓ Excellent: Modular code with separate files (cleaners.py, modelers.py, etc.)
- ✓ Logical workflow across multiple notebooks
- ✓ Reusable functions properly organized
- Some redundancy between notebooks

2. Readability & Style (4/6 pts)

- ✓ Clean, readable code with good variable names
- ✓ Consistent style throughout
- Limited code comments within notebooks
- Some cells could be better organized (data, data2, data3)

3. Technical Correctness (4/6 pts)

- ✓ Most code runs correctly
- ✓ Proper use of sklearn, pandas, numpy, plotly
- ✗ NameError in Linear_regression.ipynb cell 13: data4.columns should be data3.columns
- SettingWithCopyWarning in choropleth.ipynb

4. Library & Tool Usage (5/5 pts)

- ✓ Appropriate use of pandas, sklearn, statsmodels
- ✓ Plotly for interactive choropleth maps
- ✓ Matplotlib/seaborn for static visualizations

Documentation & Communication (17/20 pts)

1. Notebook Narrative & Explanations (5/8 pts)

- ✓ Markdown cells present with section headers
- Limited narrative: README is excellent but notebooks lack detailed explanations
- Missing interpretations directly in notebooks
- No discussion of why features were dropped

2. Visualizations (6/6 pts)

- ✓ Excellent variety: heatmaps, histograms, scatter matrices, choropleth maps
- ✓ Clear labels and titles on all plots
- ✓ Appropriate color schemes
- ✓ Interactive choropleth maps add significant value

3. Key Findings & Conclusions (6/6 pts)

- ✓ Outstanding README: clear, comprehensive, publication-quality
- ✓ Well-articulated conclusions about model performance
- ✓ Thoughtful discussion of most important predictors
- ✓ Honest assessment of limitations and future improvements

Reproducibility & Best Practices (11/15 pts)

1. Project Organization (5/5 pts)

- ✓ Clear directory structure (notebooks/, data/, src/, images/)
- ✓ Separated code from data
- ✓ Modular helper functions in src/

2. Reproducibility (3/5 pts)

- ✓ All data files included
- ✓ Saved predictions for further analysis
- ✗ NO requirements.txt or environment.yml file
- ✗ NO installation instructions in README
- Hard-coded relative paths (../) throughout notebooks

3. README & Documentation (5/5 pts)

- ✓ Exceptional README with table of contents
- ✓ Detailed methodology section

✓ Visualizations embedded and citations included

STRENGTHS

- 1. OUTSTANDING PROJECT ORGANIZATION:** The modular code structure with separate helper modules (cleaners, explorers, modelers, manipulators) demonstrates professional-level software engineering practices rarely seen in beginner projects.
- 2. COMPREHENSIVE DATA SCIENCE WORKFLOW:** Complete pipeline from data cleaning → EDA → feature engineering → multiple models → validation → results interpretation. Shows true understanding of the entire DS process.
- 3. EXCELLENT VISUALIZATIONS:** The choropleth maps are particularly impressive, showing geographic voting patterns. The correlation heatmaps, scatter matrices, and comparative histograms all effectively support the analysis.
- 4. THOUGHTFUL STATISTICAL ANALYSIS:** Used statsmodels to extract p-values and coefficients, showing understanding beyond just sklearn. The iterative feature reduction based on correlation analysis demonstrates statistical thinking.
- 5. PUBLICATION-QUALITY README:** The README is exceptionally well-written with clear explanations, embedded visualizations, proper citations, and thoughtful discussion of results and limitations. This alone sets the project apart.

AREAS FOR IMPROVEMENT

- 1. FIX RUNTIME ERROR:** In Linear_regression.ipynb cell 13, change 'index=data4.columns' to 'index=data3.columns'. This prevents the notebook from running end-to-end without error. Testing notebooks before submission is critical.
- 2. ADD DEPENDENCY MANAGEMENT:** Create a requirements.txt file with all necessary libraries and versions. Add installation instructions to the README. This is essential for reproducibility - others cannot reliably run your code without knowing exact dependencies.
- 3. ADD MORE NARRATIVE IN NOTEBOOKS:** While your README is excellent, the notebooks themselves need more markdown cells explaining why you chose specific features to drop, interpretation of visualizations, discussion of model results, and your thought process at each decision point.
- 4. VALIDATE LINEAR REGRESSION ASSUMPTIONS:** For linear regression, check residual plots to verify homoscedasticity, Q-Q plots to check normality of residuals, and discuss whether assumptions are met.
- 5. REDUCE NOTEBOOK REDUNDANCY:** The RandomForestRegressor.ipynb notebook largely duplicates work from Linear_regression.ipynb. Consider consolidating or clearly differentiating their purposes.

RED FLAGS

NONE - No critical issues found.

Minor issues noted:

- ✓ Code mostly runs (except one NameError - easily fixable)
- ✓ Statistical methodology is sound
- ✓ No plagiarism indicators (original analysis and code)
- ✓ Results match claims
- ✓ All requirements met
- Reproducibility impacted by missing requirements.txt

STUDENT-FACING FEEDBACK

Excellent work on this project! Your end-to-end workflow demonstrates a strong grasp of the data science process, from thoughtful data cleaning through multiple modeling approaches to insightful interpretation. The modular code organization and professional README are particularly impressive. Your use of choropleth maps and comprehensive EDA shows creativity and technical skill.

To improve: (1) fix the NameError in the Linear_regression notebook (cell 13), (2) add a requirements.txt file and installation instructions for reproducibility, (3) add more explanatory markdown cells within your notebooks to document your thinking, and (4) consider checking linear regression assumptions with residual plots.

Overall, this is graduate-level work for a beginner student!

FINAL GRADE SUMMARY

Category	Points
Technical Data Science Skills	38/40
Code Quality	20/25
Documentation & Communication	17/20
Reproducibility & Best Practices	11/15
SUBTOTAL	86/100

BONUS POINTS (+5): Awarded for exceptional README quality, professional code organization beyond requirements, and advanced interactive visualizations.

FINAL GRADE: A- (91/100)

ADDITIONAL NOTES

This student clearly has strong potential in data science. The project demonstrates systems thinking, statistical sophistication, communication skills, and technical versatility.

Recommendation:

This student is ready for more advanced coursework and would benefit from projects involving more complex feature engineering, hyperparameter tuning, time series forecasting, or deep learning applications. The main growth area is in documenting the analytical thought process directly in notebooks and ensuring reproducibility through proper dependency management.